

Perguntas frequentes

Preliminares

Conteúdo autoral derivado do lesson model aprovado; a bibliografia é complementar.

Pergunta: Em que tipo de projeto vale prestar atenção a Razões para estudar conceitos de LP?

Resposta: O estudo de conceitos de linguagens de programação amplia a capacidade de expressar ideias, fundamenta a escolha de linguagens adequadas, facilita a aprendizagem de novas linguagens, melhora a compreensão da implementação e favorece o uso das linguagens já conhecidas.

Fontes: presentation, slide 3, paraphrase; sebesta-11ed, p. 46, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Domínios de Programação?

Resposta: São campos de aplicação com necessidades computacionais distintas. O material apresenta aplicações científicas, comerciais, de inteligência artificial, de programação de sistemas e Web, associando cada domínio a tipos de dados, estruturas, requisitos ou linguagens características.

Fontes: presentation, slide 4, paraphrase; sebesta-11ed, p. 46, paraphrase

Pergunta: De que forma analisar Critérios de Avaliação de Linguagem pode melhorar uma decisão profissional?

Resposta: São referências para examinar e comparar linguagens. Os critérios centrais apresentados são legibilidade, capacidade de escrita, confiabilidade e custo; portabilidade e generalidade aparecem como critérios adicionais.

Fontes: presentation, slide 5, paraphrase; presentation, slide 20, paraphrase

Pergunta: Que outros critérios precisam ser considerados junto de Legibilidade?

Resposta: É a facilidade com que programas podem ser lidos e escritos. É favorecida pela simplicidade geral, por uma quantidade gerenciável de características, pela ortogonalidade, por instruções de controle adequadas, por tipos de dados claros e por escolhas sintáticas compreensíveis.

Fontes: presentation, slide 5, teacher_provided; presentation, slide 6, paraphrase; presentation, slide 10, paraphrase

Pergunta: Que consequência prática pode surgir quando a equipe ignora Ortogonalidade?

Resposta: É a possibilidade de combinar os componentes básicos de uma linguagem de programação sem restrições indevidas. Restrições especiais e tratamentos diferentes para construções semelhantes reduzem a ortogonalidade e podem prejudicar a legibilidade e a capacidade de escrita.

Fontes: presentation, slide 7, paraphrase; presentation, slide 11, paraphrase

Pergunta: Em que tipo de projeto vale prestar atenção a Capacidade de escrita?

Resposta: É a facilidade com que uma linguagem pode ser usada para criar programas destinados a um domínio de problema específico. Simplicidade, ortogonalidade, suporte para abstração e expressividade contribuem para esse critério.

Fontes: presentation, slide 5, teacher_provided; presentation, slide 11, paraphrase; presentation, slide 13, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Suporte para abstração?

Resposta: É a capacidade de definir e depois utilizar estruturas ou operações complexas sem precisar considerar continuamente grande parte de seus detalhes internos.

Fontes: presentation, slide 12, paraphrase

Pergunta: De que forma analisar Expressividade pode melhorar uma decisão profissional?

Resposta: É a disponibilidade de formas convenientes para especificar computações. Construções expressivas podem representar de maneira compacta operações que exigiriam formas mais extensas.

Fontes: presentation, slide 13, paraphrase

Pergunta: Que outros critérios precisam ser considerados junto de Confiabilidade?

Resposta: É a propriedade de um programa comportar-se de acordo com suas especificações em todas as condições consideradas. Verificação de tipos, manipulação de exceções, controle de aliasing, legibilidade e facilidade de escrita influenciam esse critério.

Fontes: presentation, slide 5, teacher_provided; presentation, slide 17, paraphrase; sebesta-11ed, p. 31, paraphrase

Pergunta: Que consequência prática pode surgir quando a equipe ignora Verificação de tipos?

Resposta: Consiste em testar a existência de erros de tipos por meio do compilador ou durante a execução. A detecção em tempo de compilação é indicada no material porque antecipa o erro e tende a reduzir o custo de correção.

Fontes: presentation, slide 14, paraphrase; sebesta-11ed, p. 30, paraphrase

Pergunta: Em que tipo de projeto vale prestar atenção a Manipulação de exceções?

Resposta: É a capacidade de interceptar erros ocorridos durante a execução, aplicar medidas corretivas e permitir que o programa prossiga quando isso for possível.

Fontes: presentation, slide 15, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Aliasing (apelidos)?

Resposta: É a possibilidade de dois ou mais nomes ou métodos de acesso referenciarem a mesma célula de memória. Uma alteração realizada por um acesso pode afetar o valor observado pelos demais, o que exige atenção quanto à confiabilidade.

Fontes: presentation, slide 16, paraphrase; sebesta-11ed, p. 31, paraphrase

Pergunta: De que forma analisar Custo pode melhorar uma decisão profissional?

Resposta: É o custo final associado à linguagem, abrangendo treinamento, aprendizado, escrita, compilação, sistema de implementação, execução, falhas decorrentes de baixa confiabilidade e manutenção. A legibilidade influencia especialmente o custo de manutenção.

Fontes: presentation, slide 18, paraphrase; presentation, slide 19, paraphrase; sebesta-11ed, p. 32, paraphrase

Pergunta: Que outros critérios precisam ser considerados junto de Portabilidade e generalidade?

Resposta: Portabilidade é a facilidade de mover programas de uma implementação para outra. Generalidade é a aplicabilidade de uma linguagem a uma ampla faixa de aplicações.

Fontes: presentation, slide 20, paraphrase

Pergunta: Que consequência prática pode surgir quando a equipe ignora Influências no Projeto de Linguagens?

Resposta: O projeto de linguagens é influenciado principalmente pela arquitetura de computadores e pelas metodologias de projeto de software. A primeira favoreceu estruturas próximas ao funcionamento da máquina; as novas metodologias estimularam novos paradigmas e recursos de linguagem.

Fontes: presentation, slide 21, paraphrase; sebesta-11ed, p. 46, paraphrase

Pergunta: Em que tipo de projeto vale prestar atenção a Arquitetura de von Neumann?

Resposta: É uma arquitetura na qual dados e programas ficam armazenados na memória, separada do processador, e instruções e dados são transmitidos para a CPU. Essa organização fundamenta linguagens imperativas, nas quais variáveis modelam células de memória, atribuições representam transferências de dados e iterações são eficientes.

Fontes: presentation, slide 22, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Ciclo de busca-decodifica-executa?

Resposta: É o ciclo de processamento no qual o contador de programa indica uma instrução, a instrução é buscada, o contador é incrementado, a instrução é decodificada e então executada, repetindo-se o processo.

Fontes: presentation, slide 24, paraphrase

Pergunta: De que forma analisar Metodologias de Projeto de Software pode melhorar uma decisão profissional?

Resposta: A evolução das metodologias deslocou o foco da eficiência das máquinas para a eficiência dos programadores, favoreceu programação estruturada, projeto top-down e refinamento passo a passo, avançou para abstração de dados e chegou à programação OO com abstração de dados, herança e polimorfismo.

Fontes: presentation, slide 25, paraphrase; sebesta-11ed, p. 35, paraphrase

Pergunta: Que outros critérios precisam ser considerados junto de Categorias de Linguagens?

Resposta: As linguagens são apresentadas nas categorias imperativas, funcionais, lógicas e de marcação/programação híbrida. Imperativas destacam variáveis, atribuições e iterações; funcionais aplicam funções a parâmetros; lógicas são baseadas em regras; e as híbridas estendem linguagens de marcação com suporte à programação.

Fontes: presentation, slide 26, paraphrase

Pergunta: Que consequência prática pode surgir quando a equipe ignora Trade-off no projeto de linguagem?

Resposta: É uma situação em que melhorar um critério pode prejudicar outro. O material destaca conflitos entre confiabilidade e custo de execução, legibilidade e capacidade de escrita, e flexibilidade de escrita e confiabilidade.

Fontes: presentation, slide 27, paraphrase

Pergunta: Em que tipo de projeto vale prestar atenção a Métodos de Implementação?

Resposta: São abordagens usadas para executar programas escritos em uma linguagem. O material apresenta compilação, interpretação pura e sistemas de implementação híbridos, além de detalhar sistemas Just in Time.

Fontes: presentation, slide 28, paraphrase; presentation, slide 36, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Compilação?

Resposta: É a tradução de um programa de alto nível em código de máquina. O processo apresentado inclui análise léxica, análise sintática, análise semântica e geração de código, com tradução comparativamente lenta e execução rápida.

Fontes: presentation, slide 30, paraphrase

Pergunta: De que forma analisar Interpretação pura pode melhorar uma decisão profissional?

Resposta: É a execução do programa por um interpretador sem uma etapa prévia de tradução. Facilita a apresentação imediata de erros em tempo de execução, mas possui execução mais lenta e geralmente requer mais espaço.

Fontes: presentation, slide 32, paraphrase

Pergunta: Que outros critérios precisam ser considerados junto de Sistemas de Implementação Híbridos?

Resposta: Combinam tradução e interpretação: o código de alto nível é traduzido para uma linguagem intermediária, que é então interpretada. A decodificação prévia torna essa abordagem mais rápida do que a interpretação pura.

Fontes: presentation, slide 34, paraphrase; sebesta-11ed, p. 43, paraphrase

Pergunta: Que consequência prática pode surgir quando a equipe ignora Sistemas de Implementação Just in Time?

Resposta: Traduzem inicialmente o programa para uma linguagem intermediária e, durante a execução, compilam métodos desse formato para linguagem de máquina. O código de máquina produzido é preservado para chamadas posteriores; por isso, o material caracteriza JIT como compilação atrasada.

Fontes: presentation, slide 36, paraphrase; sebesta-11ed, p. 43, paraphrase

Pergunta: Em que tipo de projeto vale prestar atenção a Pré-processadores?

Resposta: São programas que processam outro programa imediatamente antes da compilação. Suas instruções ficam embutidas no código; o material exemplifica essa função com as diretivas include e define de C.

Fontes: presentation, slide 37, paraphrase

Pergunta: Que erro comum ocorre quando uma equipe analisa Ambientes de Programação?

Resposta: São coleções de ferramentas utilizadas no desenvolvimento de software. O material cita UNIX com interfaces gráficas, Microsoft Visual Studio.NET e NetBeans, indicando diferenças de porte, linguagens suportadas e tipos de aplicação.

Fontes: presentation, slide 38, paraphrase; sebesta-11ed, p. 46, paraphrase

As formulações são autorais e não reproduzem exercícios da bibliografia.